

DRDO PS 26052 — AI/ML Adaptive Noise Cancellation (ANC) for Defence Speech

Deep Research Reference for Prototype Development

How to read this document: this has grown into a full reference across many research passes. If you're picking it up fresh or re-grounding before a work session, start at **Section 27 (Evidence Framework)** and **Section 28 (Scope Freeze — What We Are Actually Building First)** — they tell you what's proven vs. assumed, and exactly what's in vs. out of the current build. Everything before Section 27 is supporting detail for the items Section 28 freezes into scope.

1. What DRDO is actually scoring you on

Reading the problem statement closely, the expected solution has **five deliverables**, and each maps to a judged component:

#	Deliverable	What it proves
1	Scalable dataset pipeline (noisy-clean speech pairs)	You understand data generation isn't trivial — SNR control, impulsive + stationary mix, realism
2	SOTA AI/ML model for noise suppression	Architecture choice justified against latency/quality tradeoffs
3	Training framework with perceptual loss	You know PESQ/STOI aren't directly differentiable and handled it
4	Real-time inference engine on edge hardware	Actual quantization/ONNX/TensorRT work, not just a Jupyter notebook
5	Live prototype with mic/headset	Physical demo — mic in, denoised audio out, measurable latency

Target numbers stated: **SNR > 15 dB, STOI > 0.85, PESQ > 2.5**, real-time latency. These are realistic, achievable targets — current published SOTA

models on DNS-Challenge-class data comfortably exceed these (PESQ 3.0+, STOI 0.9+ is normal for a good model at moderate SNR). The bar isn't research novelty; it's a **working, low-latency, correctly-engineered pipeline**. This is a systems-integration challenge dressed as an ML challenge.

2. Core architecture decision: what model to actually build

Don't design a model from scratch — every serious team in this space converges on a small set of proven architectures. The real decision is picking the right point on the **quality vs. latency vs. compute** curve.

2.1 The landscape (in order of relevance to you)

DeepFilterNet2 / DeepFilterNet3 — strongest recommendation for your prototype.

<cite index="14-1">It compresses input features using equivalent rectangular bandwidth (ERB) filter banks and uses depthwise separable convolutions, achieving enhancement performance comparable to large-scale models with substantially reduced computational complexity</cite>. <cite index="18-1">Ablation experiments show the deep filtering stage boosts SI-SDR by roughly 2.8 dB and PESQ by about 0.24 MOS over ERB-only gains, outperforming complex-masking baselines like PercepNet and DCCRN at a fraction of the computational cost</cite>. <cite index="18-1">The embedded-device variant (DF2) uses grouped linear layers, depthwise separable convolutions, and minimal temporal kernels, halving real-time-factor and model footprint</cite>. It's specifically built for full-band audio on constrained hardware — exactly your use case. Open source, actively maintained (a DPDFNet variant with dual-path RNN was published as recently as this cycle to further boost it).

DCCRN (Deep Complex Convolution Recurrent Network) — the historically dominant real-time model.

<cite index="13-1">DCCRN extends the convolution recurrent network (CRN) with a complex-valued structure and achieved superior MOS scores in the Interspeech 2020 Deep Noise Suppression Challenge</cite>. <cite index="12-1">It simulates complex-valued operations inside the convolution recurrent network and won first place in the DNS challenge's real-time track</cite>. Good phase-aware baseline, well documented, lots of reference implementations — useful if you want a strong comparison point against DeepFilterNet.

Conv-TasNet — the lowest-latency option, but weaker for stationary defence noise.

<cite index="13-1">Conv-TasNet has a significantly smaller model size and much shorter minimum latency, making it suitable for both offline and real-time speech separation</cite>, and <cite index="16-1">causal Conv-TasNet with cumulative layer normalization can achieve latency as low as 3 ms</cite>. However <cite index="12-1">a low-latency variant of Conv-TasNet does not satisfy full real-time requirements at the DNS-challenge quality bar</cite> — it trades quality for speed more aggressively than DCCRN/DeepFilterNet. Worth mentioning in your report as the "ultra-low-latency" alternative you evaluated.

FullSubNet / Inter-SubNet — best pure-quality option if Jetson compute is genuinely not a constraint.

<cite index="14-1">FullSubNet separately models full-band and sub-band features, fusing global contextual information with local spectral detail to significantly improve speech-noise discrimination</cite>. Heavier than DeepFilterNet2, but a good "ceiling" model to benchmark against in your report even if you don't deploy it.

DTLN — smallest footprint option worth knowing about.

<cite index="12-1">DTLN combines the STFT operation and a learned analysis/synthesis basis into a stacked network with fewer than one million parameters, and is capable of real-time processing</cite>. If Jetson power/latency margins get tight, this is your fallback.

2.2 Recommendation

Primary model: DeepFilterNet2/3 as your main deliverable — it's purpose-built for exactly this problem (full-band, low-complexity, real-time, embedded).

Secondary/comparison model: DCCRN, trained on the same data, reported in your results table for the "we evaluated multiple SOTA architectures" credibility that DRDO evaluators look for. This gives you a strong report narrative without doubling your engineering burden, since both operate in the complex/masking domain and share a lot of data pipeline code.

3. Datasets — the actual hard part of this problem statement

The problem explicitly separates **stationary**, **non-stationary**, and **impulsive** defence noise. Generic speech-enhancement datasets (DNS Challenge,

VCTK+DEMAND) give you the first two categories well but are weak on realistic defence-specific impulsive/military content. You need to combine sources.

3.1 Clean speech + base noise (the backbone)

[\[20\]](#) The DNS Challenge (Interspeech 2020) dataset consists of a clean speech set with about 500 hours of clips from 2,150 speakers, and a noise dataset with over 180 hours of clips across 150 classes. [\[21\]](#) The later DNS4-generation clean speech set spans 562+ hours across six languages, and [\[23\]](#) its noise data draws from AudioSet, Freesound, and DEMAND. This is your foundation — it already ships with an open-sourced dynamic-mixing data generation pipeline, so you don't have to build the mixing logic from scratch.

Standard mixing recipe used across nearly every paper above: [\[20\]](#) clean speech is mixed at a randomly selected SNR between -5 and 20 dB, with roughly 75% of clips convolved with a room impulse response (RIR) first for reverberant realism. [\[22\]](#) SNR levels for the official DNS synthesis pipeline are sampled uniformly between 0 and 40 dB, with mixed signal RMS set between -15 and -35 dBFS. Your problem statement doesn't demand extreme low-SNR (below -5 dB) robustness explicitly, but gunfire/artillery in the near field will produce very low instantaneous SNR — plan your sampling range accordingly (recommend -10 to +20 dB instead of the default DNS range, given the domain).

3.2 Defence/military-specific noise — this is what differentiates your submission

This is the part most teams will skip or fake with generic noise, and it's exactly where you can score points for realism:

- **MAD (Military Audio Dataset)** — [\[30\]](#) a purpose-built military audio dataset with classes covering communication, gunshot, footsteps, shelling, vehicle, helicopter, and fighter jet audio, containing 8,075 samples across roughly 12 hours, extracted from real military training/education videos rather than games or simulations. This is your single best source — directly matches "gunshots, artillery fire, helicopter rotor noise, armored vehicle sound" from the problem statement. Available on Figshare per the paper.
- **Gunshot-specific corpora** — the Certus Caliber Classification Gunshot Dataset (C3GD) offers [\[32\]](#) clean, minimally-clipped gunshot recordings across a range of calibers, microphones and

platforms, collected in outdoor field settings — useful as an impulsive-noise-only source you layer onto clean speech yourself at controlled SNR (since the recordings are clean, you control the mixing). UrbanSound8k's gun_shot class is also usable as filler/augmentation but less realistic for actual firearms.

- **NOISEX-92** — classic but still relevant: factory and f16 (military aircraft) noise from NOISEX-92 has historically been used as unseen-noise test conditions in speech enhancement papers, alongside babble noise. Good for out-of-distribution generalization testing, i.e., proving your model wasn't just overfit to your training noise set.
- **Vehicle/ground-sensor audio** — a military vehicle acoustic dataset recorded at Twenty-Nine Palms, CA in 2001 captured sound, seismic and infrared sensor data as a dragon wagon and assault amphibian vehicle drove through a sensor field, at a 4960 Hz sample rate — niche and low sample rate, but a legitimate citeable source for "armored vehicle sound" if you want to claim domain coverage in your report.

3.3 Practical dataset construction plan

1. Base: DNS Challenge clean speech + noise (backbone, gives you volume and a proven mixing recipe).
2. Layer in MAD dataset classes (gunshot, shelling, helicopter, vehicle, fighter) as a **defence-noise augmentation set**, mixed at your extended SNR range.
3. Use C3GD or similar clean gunshot recordings for controlled **impulsive-noise injection** — this needs separate handling from stationary noise because impulsive events are short, high-amplitude, and shouldn't just be additively mixed at a flat SNR across the whole utterance (you want randomized onset timing and per-event amplitude, not a fixed global SNR).
4. Reserve NOISEX-92 (babble, f16, factory) purely as a **held-out generalization test set** — never train on it, use it in your final report to show the model generalizes beyond its training noise distribution. This is a strong point in a DRDO evaluation.
5. Augmentation: random noise mixing at varied RMS/SNR, plus reverberation via RIR convolution as your two baseline augmentations; add clipping simulation and mic-frequency-response coloring if time allows, since the problem statement explicitly names "clipping" as an expected augmentation.

4. Training pipeline: features, targets, loss functions

4.1 Input representation

The problem statement wants both spectrogram-based and raw-waveform approaches considered, plus complex-domain phase preservation.

Practically:

- Use **STFT-based complex-domain masking** (DCCRN/DeepFilterNet style) as your primary path — operating on complex-valued convolution structures preserves phase information, which matters a lot for impulsive noise (gunshots have sharp phase discontinuities that magnitude-only masking destroys).
- DeepFilterNet's ERB-compressed features specifically reduce compute versus full STFT bins while retaining perceptual relevance — this is the efficiency lever that makes real-time embedded deployment feasible.

4.2 Loss functions

Don't use a single loss — every strong paper in this space combines several:

- **SI-SNR (scale-invariant SNR)** as the anchor loss — across joint-loss experiments, SI-SDR consistently plays the key role in training, with STOI or PMSQE added as secondary terms improving specific metrics without hurting the SI-SDR-driven structure.
- **PMSQE (Perceptual Metric for Speech Quality Evaluation)** as a differentiable PESQ proxy — PMSQE is a perceptual metric derived from the PESQ algorithm, computed per-frame from the power spectra of reference and processed signals, specifically designed to be usable as a training loss for deep learning speech enhancement, and achieved an average 0.12-point PESQ improvement over other state-of-the-art metrics on unseen noise.
- **Differentiable STOI loss** as a third term when intelligibility is the priority — but note a caution: a controlled study found that while DNN systems trained on STOI/PMSQE/SI-SDR-family losses show strong objective metric gains, these gains are sometimes contradicted by actual subjective intelligibility tests. Don't over-rely on the metric alone — do a few informal listening checks yourself before the demo, not just trust the STOI number.
- Simple combination that's well-validated: $L = \text{SI-SNR} + 0.5 \cdot \text{PMSQE}$ or $L = \text{SI-SNR} + 0.5 \cdot \text{STOI_loss}$, per the joint-loss results cited above.

4.3 Evaluation metrics (what you report)

- **SNR / SI-SNR** — target >15 dB per problem statement.
 - **STOI** (0 – 1 scale) — target >0.85.
 - **PESQ** (-0.5 to 4.5, or 1.0-4.5 for wideband) — target >2.5. Note PESQ has narrowband (NB) and wideband (WB) variants; report which one you use.
 - Report all three **per noise category** (stationary / non-stationary / impulsive) separately, not just averaged — this directly demonstrates you handled the problem statement's three-way distinction, which is a specific ask DRDO called out by name.
-

5. Real-time embedded deployment — the part that actually distinguishes a working prototype from a notebook

5.1 Target hardware

The problem statement names <cite index="60-1">NVIDIA Jetson AGX Orin 64GB, which uses the NVIDIA Ampere architecture with 2048 CUDA cores and a 12-core ARM CPU, delivering up to 275 TOPS of AI compute with 64GB of high-bandwidth memory</cite> as an example platform. <cite index="61-1">Specifically it has 2048 CUDA cores and 64 Tensor cores, a 12-core Arm Cortex-A78AE CPU, 3MB L2 + 6MB L3 cache, 64GB LPDDR5 memory at 204.8 GB/s bandwidth</cite>, and <cite index="65-1">power is configurable from 15W to 60W</cite>. This is enormous headroom for a speech enhancement model — DeepFilterNet2/DCCRN-class models are a few million parameters and run in real time on a laptop CPU, let alone a 275-TOPS SoC. Your bottleneck won't be compute; it'll be **I/O latency, buffering, and correct real-time streaming implementation**. If you don't have Orin access, an AGX Xavier, Orin Nano, or even a decent laptop is enough to prototype the model — just be explicit in your report about the target vs. dev hardware.

5.2 Optimization pipeline (PyTorch → deployable)

Standard path, well-validated across many embedded audio/vision papers:

1. Train in PyTorch.
2. Export to **ONNX**. <cite index="10-1">Export the trained model to ONNX for interoperability, then run it on the Jetson via TensorRT, which is NVIDIA's

automatic model optimization and acceleration tool for neural network inference

3. Convert ONNX → **TensorRT engine**. Across comparative benchmarks on resource-limited hardware, ONNX-to-TensorRT conversion consistently delivers the best latency and throughput performance, with TensorRT reported to deliver 81%-391% performance improvement over baseline GPU inference in some studies.
4. **Precision reduction**: enable mixed-precision execution (FP32/FP16) during TensorRT engine generation, manually pinning any layers whose parameter ranges exceed FP16 back to FP32. For audio models specifically, precision reduction to 16-bit fixed-point has been shown to halve model memory footprint without compromising objective speech quality in embedded DNN speech-enhancement deployments.
5. **INT8 quantization** if you want to push further — calibrate with a representative dataset sample using static-range calibration on a representative dataset subset, which is the standard TensorRT/ONNX quantization procedure for both microcontroller-class and Jetson-class deployment targets.

5.3 Latency budget — a concrete reference point

This matters for your report's credibility. A comparable embedded time-domain speech enhancement model deployed on an FPGA achieved 9.7 ms first-sample latency for denoising, meeting the 10 ms clinical hearing-aid threshold, while a heavier speech separation model reached 16.0 ms, and DCCRN's algorithmic latency runs around 62.5 ms, acceptable for teleconferencing but not for a hearing-aid-grade real-time requirement — while causal Conv-TasNet variants have achieved latencies as low as 3 ms. For a defence radio/comms use case, you want to be well under 40 ms end-to-end (algorithmic + inference + I/O), ideally targeting 10 – 20 ms. State your measured number explicitly in the demo rather than a theoretical one — judges will ask.

5.4 Reference implementations worth cloning/studying directly

- **DeepFilterNet** official repo (Rust core + Python training, exports to ONNX) — closest match to "state of the art + embedded-ready" that DRDO is asking for.
- A public example of exactly this deployment pattern already exists: a GitHub project ("jetson_denoiser") forks a real-time speech

enhancement model specifically to make it deployable on an embedded Jetson Nano — worth reviewing as a working reference for the ONNX/TensorRT plumbing even though your target model differs.

6. The hybrid AI + adaptive filter layer (optional but scores well)

The problem statement explicitly says the system "can optionally include a lightweight adaptive filter (e.g., LMS) for residual noise suppression" — this is a chance to show depth beyond a pure black-box DNN.

- Classic baseline: the filtered-x LMS (FxLMS) algorithm accounts for the time delay of the secondary acoustic path in a typical feedforward/feedback ANC system with a reference mic and error mic, and the LMS family is favored for its simplicity, robustness, and low computational load.
 - The current SOTA hybrid pattern is **SFANC-FxNLMS**: a secondary-path-decoupled ANC approach uses two time-domain convolutional recurrent networks — one modeling the nonlinear secondary path, one modeling its reverse — to compute a decoupled error signal, with the actual control signal generated by an adaptive filter minimizing that error. The hybrid SFANC-FxNLMS combination provides fast convergence speed and continuous updating ability — a 1D CNN classifies/selects the right pre-trained control filter per audio frame, and FxNLMS fine-tunes it in real time, giving you both the adaptability of DNNs and the low-compute robustness of classical LMS.
 - Practical framing for your prototype: use the **DNN (DeepFilterNet/DCCRN) as your primary suppression stage**, and add a **lightweight LMS/NLMS stage post-DNN** to mop up residual stationary hum/hiss the DNN under-suppresses. This is cheap to implement (a few dozen lines of NLMS), directly answers the "optional adaptive filter" ask, and gives you a nice ablation result (DNN-only vs. DNN+LMS) for your report.
-

7. Physical prototype integration

- **Mic setup**: problem statement wants primary + reference microphone — use a two-mic setup (e.g., a USB or I2S dual-mic array) so you can genuinely demonstrate reference-noise-aware suppression, not just

single-channel denoising. This also lets you implement true feedforward ANC (reference mic picks up noise before it reaches the primary mic) rather than post-hoc denoising alone — a meaningfully stronger demo.

- **Output:** headset/comms unit playback of the enhanced signal, with the raw noisy signal available on a toggle for live A/B comparison during judging — this single UX decision (a switch between raw/enhanced audio) is disproportionately persuasive in a live demo.
- **Streaming implementation:** don't process full utterances — implement true frame-by-frame streaming with causal buffering (this is where DeepFilterNet's causal design and DCCRN's frame-online mode matter — verify you're using the causal/streaming inference mode of whichever model you pick, not the offline batch mode used during training evaluation).
- **Live latency measurement:** instrument mic-in to speaker-out latency directly (e.g., loopback click test) and report the actual number — don't just cite the model's theoretical algorithmic latency.

8. Suggested build sequence (given a hackathon-style timeline)

1. **Data pipeline first** (highest-leverage, most-skipped step): get DNS Challenge + MAD dataset mixing working end-to-end, producing labeled noisy/clean pairs across your three noise categories, before touching model code.
2. **Train DeepFilterNet2 baseline** on your combined dataset — this is the fastest path to a working, real-time-capable model since pretrained checkpoints and training scripts already exist; fine-tune rather than train from scratch given your timeline.
3. **Evaluate:** SNR/STOI/PESQ per noise category, plus the NOISEX-92 held-out generalization test.
4. **Export → ONNX → TensorRT**, benchmark latency on your actual target hardware.
5. **Add the NLMS residual stage** if time permits — cheap addition, strong report content.
6. **Wire up live mic/headset demo** with the raw/enhanced toggle, and instrument real end-to-end latency.
7. **(Stretch)** Train a second model (DCCRN) on the same pipeline purely for the comparison table in your report — costs little extra given the shared

data pipeline.

9. Peer-reviewed benchmark numbers (academic literature cross-check)

The following pulls concrete, citable performance numbers from peer-reviewed papers to replace some of the general claims above with hard evidence you can put directly in a DRDO report.

9.1 Complex-domain / phase-aware models vs. magnitude-only baselines

A deep complex convolutional model (DCCNN) tested specifically on **unseen non-stationary noise** — the hardest generalization case, closest to your "unknown defence noise" scenario — showed a perceptual quality gain of roughly 0.44 MOS points over state-of-the-art single-stage techniques, an SNR improvement of over 3 dB, and a 0.2 STOI improvement versus baselines, with the intelligibility gains holding specifically in low-SNR conditions [[Iqbal et al. 2024](#)]. This is direct evidence that phase-aware complex-domain processing (which DeepFilterNet and DCCRN both use) matters most exactly where the problem statement is hardest — unseen, non-stationary defence noise.

9.2 DCCRN's successor and a small-footprint variant worth knowing about

DCCRN+ extends the original DCCRN with sub-band processing (bands split/merged by learnable filters instead of hand-engineered FIR filters), a complex TF-LSTM for better time-frequency temporal modeling, and an added a-priori SNR estimation module, surpassing the original DCCRN and several competing models on PESQ and DNSMOS in the Interspeech 2021 DNS Challenge [[Lv et al. 2021](#)]. If your baseline DCCRN result underwhelms, DCCRN+ is the direct upgrade path.

Separately, **Distil-DCCRN** uses knowledge distillation (a larger "teacher" model transferring both output and intermediate-layer knowledge to a smaller "student") to shrink DCCRN to just 30% of its original parameter count while still outperforming the full DCCRN on PESQ and SI-SNR on the DNS test set [[Han et al. 2024](#)]. Worth citing in your report as the answer to "how would you shrink this further for a smaller embedded target than Jetson."

9.3 Extreme low-SNR performance (directly relevant to near-field gunfire/artillery)

A systematic study of single-channel deep learning models for drone ego-noise — chosen because it's one of the few domains that regularly tests speech enhancement at **-30 to 0 dB SNR**, comparable to a near-field gunshot or artillery blast overwhelming a mic — found that time-frequency complex-domain and UNet encoder-decoder architectures outperformed other approaches, with the best model (a complex-domain UNet) improving ESTOI from 0.1 to 0.4, PESQ from 1.0 to 1.9, and SI-SDR from -15 dB to +3.7 dB at an input SNR of -15 dB [\[Mukhutdinov et al. 2023\]](#). This is a useful calibration point: at genuinely extreme SNR (near-field gunfire), don't expect to hit PESQ > 2.5 — the DRDO target numbers are realistic for moderate-to-severe noise, not point-blank impulsive events, and it's worth stating that scope boundary explicitly in your report rather than overselling.

9.4 Transient/impulsive noise on real embedded hardware — a concrete latency number

A real-time single-channel enhancement algorithm specifically targeting **both stationary and transient noise** (tested at -10 to +10 dB SNR across 115 environmental sounds) reported a measured hardware runtime of **4.3 ms to process a 10 ms frame** — i.e., faster than real time with clear margin — while improving speech quality indicators by 5 – 7.8% and STOI by ~2 – 2.4% over baseline algorithms, and outperforming a comparable RNN-based method specifically on transient noise suppression [\[Liang et al. 2020\]](#). This is a good real-world reference point for the "processing 1 second of audio in well under 1 second" real-time factor you'll want to report from your own Jetson benchmarking.

9.5 Deep filtering's known weakness — dereverberation

Worth flagging honestly in your report rather than having a judge catch it: DeepFilterNet is excellent at noise reduction but has a structurally limited dereverberation capability, and recent work proposes a specific architectural extension to address this gap while keeping DeepFilterNet's real-time, resource-constrained efficiency [\[Rosenbaum et al. 2025\]](#). If your demo environment has significant echo/reverberation (e.g., an indoor room rather than open field), this is the one weakness to proactively mention and mitigate (e.g., a short dereverberation pre-stage, or simply controlling the demo acoustics).

9.6 Quantization/pruning — realistic speedup numbers for your optimization stage

Across recent edge-deployment literature: a pruning + post-training quantization + ONNX pipeline on Jetson-class hardware reduced inference latency by up to 3.5x and memory usage by 2.8x while retaining over 92% of perceptual quality (MOS-based) [[Meenakshisundaram et al. 2025](#)]. A hybrid sensitivity-aware pruning + INT8 quantization framework tested specifically across heterogeneous NVIDIA Jetson platforms achieved up to 3.12x inference speedup and 55% model size reduction while holding accuracy drop under 1.5% [[Gopalan et al. 2026](#)]. Given your model (DeepFilterNet2-class, a few million parameters) is already tiny relative to Jetson AGX Orin's 275 TOPS budget, you likely won't need to push quantization this aggressively — but these numbers are useful ammunition if a judge asks "what if you had to run this on a smaller SoC than Orin."

10. DRDO's own institutional literature on this exact problem — read this first

Before anything else, a directly on-point paper: "**Artificial Intelligence Driven Advances in Noise Cancellation: A Comprehensive Review on Overcoming Noise Hazards in Military Operations**," published in *Defence Science Journal* (DRDO's own peer-reviewed journal), Vol. 76, No. 3, May 2026, pp. 408 – 417, DOI [10.14429/dsj.21095](#). This is essentially DRDO's own current institutional framing of your exact problem statement, so it's worth reading in full and citing directly in your submission — evaluators may well be familiar with it.

Key points from the paper:

- It frames military noise explicitly as **impulsive and non-stationary**, arising from aircraft and machinery in the combat zone — the same three-way split your problem statement uses [[Narain et al. 2026](#)].
- It states plainly that traditional adaptive filters "provide a foundation for real-time noise reduction" but "perform poorly with non-linear and wideband noise" — i.e., the paper itself validates the problem statement's premise that classical LMS/Wiener approaches are insufficient alone [[Narain et al. 2026](#)].
- It distinguishes **passive noise cancellation (PNC)** — physical means like earplugs/barriers, effective at high frequencies but not low frequencies (a common issue in military battle noise) — from **active noise cancellation (ANC)**, which generates anti-noise signals and is more effective against

fast-moving noise sources typical in military settings. This PNC/ANC distinction is worth explicitly addressing in your report: your DNN pipeline is squarely in the ANC category, and you can note that a hybrid PNC (physical earcup/hearing protection) + your AI-ANC system is the realistic deployed configuration, not AI-ANC alone.

- **Concrete performance numbers directly relevant to your target metrics:** recent CRN and GAN-based noise suppression implementations reviewed in the paper achieved **SNR gains of 14 – 16 dB** and **PESQ improvements exceeding 7%** in high-noise defence conditions [\[Narain et al. 2026\]](#). This directly validates that the problem statement's SNR > 15 dB target is realistic and achievable with exactly the architecture families (CRN/GAN) you're already planning to use, and gives you a citable number to benchmark your own results against.
- It also covers integration into **communication systems, personal protective equipment, and operational platforms**, and explicitly names **latency and scalability** as the open challenges — the same operational concerns your Section 5 (embedded deployment) addresses.

Action item: cite this paper explicitly in your problem-background section — showing you found and engaged with DRDO's own recent literature on the topic is a strong signal of genuine research effort, distinct from generic ML papers.

11. Additional research: multi-mic beamforming, quality metrics, augmentation, and human-factors context

11.1 Dual/multi-microphone beamforming + DNN (directly extends your primary+reference mic setup)

The problem statement's mic setup (primary + reference) is exactly the classical beamforming configuration, and combining it with a DNN post-filter is well validated:

- A dual-microphone approach using a densely-connected convolutional recurrent network for dual-channel complex spectral mapping, compressed via structured pruning for low-latency real-time use, **consistently outperformed both an earlier dual-channel approach and a DNN-based beamformer** [\[Tan et al. 2021\]](#) — direct evidence that your

planned primary+reference mic setup benefits from complex spectral mapping over classical beamforming alone.

- The standard, well-validated architecture pattern is **beamform first, DNN-postfilter second**: a multichannel MMSE solution factorizes into an MVDR/GSC beamformer followed by a single-channel DNN-based Wiener-style post-filter, and combining supervised speech separation with adaptive beamforming this way **outperforms either method used individually** [[Šarić et al. 2022](#)]. This is the cleanest architectural pattern to adopt: MVDR/GSC beamform your primary+reference signal, then run your DeepFilterNet/DCCRN model as the post-filter.
- A useful design insight for headset-specific ANC: an adaptive beamforming system built specifically for real-time speech enhancement in hands-free headset communication combines an MVDR beamforming technique with a DTLN network implemented directly in the frequency domain — worth referencing if your final demo uses a headset form factor rather than open mics [[Pias et al. 2024](#)].

11.2 DNSMOS — the metric you'll actually want alongside PESQ/STOI for live demo scoring

PESQ and STOI both require a clean reference signal, which you won't have for live, real-world demo audio (no "clean" version of a live gunshot-corrupted recording exists to compare against). This is where **DNSMOS** becomes essential: it's a non-intrusive (no-reference) objective metric, trained to predict human Mean Opinion Score ratings directly from the processed audio without needing a clean reference signal, making it the practical choice for scoring your live demo output [[Reddy et al. 2020](#)]. A follow-up version, **DNSMOS P.835**, extends this to output three separate scores — speech quality (SIG), background noise quality (BAK), and overall quality (OVRL) — and correlates very strongly with human ratings (Pearson correlation of 0.94 for speech quality and 0.98 for background/overall quality) [[Reddy et al. 2021](#)]. **Practical recommendation**: report PESQ/STOI/SNR on your synthetic held-out test set (where you have clean references), and report DNSMOS P.835 (SIG/BAK/OVRL) on your live mic recordings during the actual demo — this is the standard way DNS-Challenge-caliber work handles the reference-vs-no-reference gap, and DNSMOS Pro is publicly available as a pip-installable model, cheap to run alongside your pipeline.

11.3 A caution on generalization — worth designing your test protocol around

A rigorous generalization study comparing FFNN, Conv-TasNet, DCCRN, and MANNER found a consistent, important pattern: **performance degrades most under speech mismatch** (unfamiliar speakers/languages), while **noise and room (reverberation) generalization is comparatively easy to achieve simply by training on multiple databases** — and, notably, models that perform best in matched (seen) conditions can become **inferior to a much simpler feedforward baseline** once conditions are mismatched [\[Gonzalez et al. 2023\]](#). Practical implication for your dataset plan (Section 3): don't over-invest in noise diversity at the expense of speaker diversity. Make sure your clean speech training set has good speaker variety (the DNS Challenge's 2,150+ speakers already gives you this), and treat your NOISEX-92 held-out generalization test primarily as a check on noise robustness — but also hold out a set of speakers never seen in training, since that's the axis most likely to hurt you at demo time if a judge or unfamiliar tester speaks into your mic.

11.4 Human-factors grounding — why the STOI/PESQ targets matter operationally

This isn't ML literature but is useful context for your report's motivation section: a study simulating a Navy Combat Information Center found that operational task performance **degraded systematically as speech intelligibility decreased**, with the sharpest performance drop occurring specifically between 80% and 60% intelligibility on a standard rhyme test — directly linking the kind of intelligibility metric your problem statement targets (STOI) to real mission-critical outcomes, not just an abstract audio-quality number [\[Keller et al. 2017\]](#). Separately, research on Canadian Armed Forces communications found that **armoured vehicle noise specifically degrades speech intelligibility** for radio and face-to-face communication alike, and that this effect is more pronounced for non-native speakers — relevant since your problem statement explicitly names armored vehicle noise as a target category [\[Nakashima et al. 2018\]](#). A one-line citation of either in your report's introduction gives the DRDO-style "why this matters operationally" framing that a pure ML report often lacks.

11.5 Data augmentation refinement — RIR simulation quality matters

For the reverberation augmentation mentioned in Section 3.3: don't default to the simplest image-source-method RIR simulator if you have compute budget. A stochastic RIR generator built specifically for audio-augmentation use (not requiring room-geometry specification) was shown, when used for speech-enhancement training, to **outperform the conventional image-source method on a wide range of metrics**, improving many of them by more than

5% [\[Masztalski et al. 2020\]](#). If your image-source RIR augmentation underperforms, this is a fast, well-documented swap.

12. Concrete software stack — exact repo, exact commands

Stop treating "pick a model" as an open design question — fork a working repo and adapt it. This is the single highest-leverage decision for a tight timeline.

12.1 The repo to fork

[Rikorose/DeepFilterNet](#) — the official implementation, actively maintained, with pretrained DeepFilterNet/DeepFilterNet2/DeepFilterNet3 checkpoints included. It ships a working CLI, a Python training entry point, and a documented ONNX export pipeline — you are not building any of this from scratch.

Inference (baseline, to confirm the pipeline works before you touch training):

```
pip install deepfilternet
python DeepFilterNet/df/enhance.py -m DeepFilterNet2
path/to/noisy_audio.wav
```

or in Python directly:

```
from df import enhance, init_df
model, df_state, _ = init_df()          # loads pretrained
DeepFilterNet2 by default
enhanced_audio = enhance(model, df_state, noisy_audio)
```

This alone gets you a working noise-suppression demo on day one — everything after this is fine-tuning on your defence-specific dataset and hardening for real-time/embedded deployment [\[Rikorose/DeepFilterNet\]](#).

Fine-tuning on your combined DNS+MAD+gunshot dataset:

```
python df/train.py path/to/dataset.cfg path/to/data_dir/
path/to/base_dir/
```

The training script auto-creates a `base_dir` for logging, audio samples, checkpoints, and config on first run — copy the config from

`DeepFilterNet/pretrained_models/DeepFilterNet` as your starting point rather than writing one from scratch [\[Rikorose/DeepFilterNet\]](#).

12.2 ONNX export — the exact mechanism

The export script (`df/scripts/export.py`) **splits the model into three separate ONNX components** — an encoder, an ERB decoder, and a DF (deep filtering coefficient) decoder — which are packaged together into a single `tar.gz` for deployment, and recombined at inference time by the runtime [\[DeepWiki: DeepFilterNet ONNX Export\]](#). Critically, the export defines **dynamic axes** on the sequence-length dimension (`feat_erb`, `feat_spec`, `e0`, `emb`, `c0`, `lsnr`), which is what allows the exported model to process audio of arbitrary length in streaming chunks rather than requiring a fixed-size input — this is the detail that makes real-time streaming inference possible at all, so don't skip or "simplify" this step when you export your fine-tuned model.

12.3 Real-time streaming inference — don't reinvent this

Two directly reusable references for the actual real-time streaming implementation (mic-in → chunked inference → speaker-out), which is usually the part teams underestimate:

- [shimondoodkin/deepfilter-rt](#) — a working real-time DeepFilterNet + ONNX Runtime implementation with measured performance: **10 ms frame size, 20 ms lookahead, average 1.70 ms and max 5.21 ms processing time per frame** on their reference hardware, using a "split streaming" mode where all GRU hidden states are passed as explicit ONNX I/O tensors between frames (this is what makes frame-by-frame streaming stateful and coherent, versus naively feeding a stateless window each time, which the repo notes is markedly slower) [\[shimondoodkin/deepfilter-rt\]](#). This single number — **1.7 ms average per 10 ms frame** — is your latency budget reference: you have roughly 6x headroom before you're not real-time anymore, even on far weaker hardware than a Jetson AGX Orin.
- [grazder's PyTorch streaming reimplementation](#) — a from-scratch PyTorch (no Rust/tract dependency) streaming implementation that exports to a single ONNX graph, useful if you want to understand or modify the streaming state machine directly in Python rather than treating it as a black box.
- [DPDFNet](#) — if your evaluation shows DeepFilterNet2 underperforming on your defence noise set, this is a documented drop-in upgrade (adds dual-path RNN blocks for stronger long-range temporal modeling) with PyTorch + ONNX + TFLite checkpoints and a working real-time demo already built.

12.4 What NOT to do

Don't hand-roll your own STFT/ISTFT framing, GRU-state buffering, or overlap-add logic for the streaming path — this is exactly the kind of "invisible" engineering work that eats a week and produces subtle bugs (clicks, drift, state corruption on buffer underrun) that are very hard to debug live at a demo table. Fork the reference streaming implementation and modify from there.

13. Hardware bill of materials — concrete part numbers

Component	Recommended option	Why
Compute	Jetson AGX Orin 64GB Dev Kit (per problem statement)	275 TOPS, 64GB LPDDR5 — massive headroom for a few-million-parameter model; your bottleneck will be I/O, not compute
Primary + reference mic	Seeed reSpeaker XVF3800 USB 4-Mic Array	Purpose-built for exactly your use case: onboard AEC, AGC, direction-of-arrival, beamforming, noise suppression and dereverberation on-chip (XMOS XVF3800), plug-and-play USB on Jetson, with I2S mode also available for lower-level raw multi-channel capture if you want your DNN to see truly raw signal rather than the onboard-DSP-preprocessed one [Seeed reSpeaker XVF3800].
Budget/DIY mic alternative	INMP441 I2S MEMS mic breakout (x2, wired as primary+reference)	Cheap, well-documented I2S digital mic, direct wiring to Jetson's I2S pins is a known-working pattern from the Jetson/ReSpeaker community, gives you full raw-signal control without an onboard DSP "cleaning" the signal before your model sees it [respeaker/get_started_with_respeaker_issue #92].

Component	Recommended option	Why
Output	Any USB or 3.5mm powered speaker / comms headset	For the raw/enhanced A-B toggle demo described in Section 7
Optional stretch	reSpeaker Lite (XMOS XU316, dual mic, I2S+USB)	Smaller/cheaper alternative if you want a second, lower-power demo unit alongside the Jetson rig

Important design decision: if you use the XVF3800's onboard beamforming/AEC, you're effectively stacking a classical DSP front-end ahead of your DNN — which is a legitimate and defensible hybrid architecture (Section 11.1's beamform-then-DNN-postfilter pattern), but you should state this explicitly in your report rather than let it look like your DNN alone is doing everything. If you want a cleaner "our DNN did the heavy lifting" story, use the raw I2S dual-mic path instead and implement your own MVDR/GSC beamforming stage in software ahead of the DNN.

14. The hybrid NLMS residual stage — exact algorithm to implement

Section 6 covered the SFANC-FxNLMS concept; here's the actual minimal-viable version you can implement in an afternoon as a post-DNN residual cleanup stage, using the standard Filtered-x NLMS (FxNLMS) update.

Filtered-x NLMS update rule (feedforward ANC using your reference mic):
For each sample index m :

1. Filter the reference signal $x[m]$ through an estimate of the secondary path $\hat{S}(z)$ to get the filtered reference $x_f[m]$.
2. Compute the adaptive filter output (control signal) $y[m] = w[m]^T \cdot x_f[m]$ and play it through the speaker/apply it to the signal path.
3. Measure the residual error $e[m]$ (from your error mic, or in the post-DNN-stage case, the DNN's own residual noise floor).
4. Update the weights using the **normalized** step size — this is what makes NLMS more stable than plain LMS, since it removes the manual mu-tuning-vs-input-power sensitivity that plain LMS has:

$$w[m+1] = w[m] + \mu \cdot (x_f[m] \cdot e[m]) / (\|x_f[m]\|^2 + \epsilon)$$

where μ is the step size (typically 0.01 – 0.5, tune empirically), and ϵ is a small constant preventing division by zero during silence [\[Study of Switched Step-size FxNLMS for ANC\]](#).

The core tradeoff to know and state in your report: a larger μ converges faster but leaves a higher steady-state residual error; a smaller μ converges slower but achieves lower final residual noise — this is a fundamental, well-documented tradeoff in every adaptive-filter reference, not something you can eliminate, only tune for your scenario [\[adaptive filter step-size tradeoff, multiple patent/reference sources\]](#). For a live demo, bias toward faster convergence (larger μ) since your noise environment changes are sudden (gunshot onset) and you need the filter to react quickly; accept a slightly higher residual noise floor as the cost.

Where to insert it: run this as a lightweight post-processing stage on the DNN's output, using your reference mic signal as $x_f[m]$ — this specifically targets stationary residual hum/hiss the DNN under-suppresses, which is exactly the gap the problem statement's "optional adaptive filter for residual noise suppression" line is pointing at. A basic NLMS implementation is ~30-40 lines of NumPy and runs orders of magnitude faster than your DNN inference, so it will never be your latency bottleneck.

15. Evaluation pipeline — exact packages and a ready-to-run script structure

```
pip install pesq pystoi mir_eval
pip install https://github.com/schmiph2/pysepm/archive/master.zip
# optional: adds SI-SDR, segmental SNR, composite scores in one
package
```

Minimal evaluation snippet (adapt to loop over your test set, per noise category):

```
from pesq import pesq
from pystoi import stoi
import numpy as np

def evaluate(reference, enhanced, sr=16000):
    pesq_score = pesq(sr, reference, enhanced, 'wb') #
```

```

wideband PESQ, use 'nb' for narrowband/telephony-style comparison
stoi_score = stoi(reference, enhanced, sr, extended=False)
si_snr = 10 * np.log10(
    np.sum(reference**2) / np.sum((reference - enhanced)**2) +
    1e-8
)
return {"PESQ": pesq_score, "STOI": stoi_score, "SI-SNR":
    si_snr}

```

Reference for library usage and expected value ranges (a working end-to-end example on real audio, useful to sanity-check your own numbers against): a torchaudio speech enhancement tutorial reports representative scores of **PESQ 2.01, STOI 0.77, SI-SNR 4.10 dB** for a baseline (non-defence-specific) enhancement example — a useful sanity floor to compare your own numbers against before you conclude your pipeline is broken or your model is bad [[PyTorch/torchaudio MVDR speech enhancement tutorial](#)]. PESQ requires 8kHz (narrowband) or 16kHz (wideband) sample rate specifically — resample your DeepFilterNet3 output (which natively runs at 48kHz) down to 16kHz before scoring, or your PESQ calls will error out [[python-pesq \(ludlows\)](#)].

For your live demo, since you won't have a clean reference for real mic audio, install and use **DNSMOS P.835** instead (see Section 11.2) — Microsoft publishes both a pip-installable local model and an Azure-hosted evaluation service; the local model is what you want for a demo where you can't rely on internet access.

Important standards note, worth knowing before a technically-informed judge raises it: ITU formally withdrew the PESQ standard (P.862, and its P.862.1–3 companions) from its catalogue on 5 January 2024, redirecting to **P.863 (POLQA)** as the current recommendation — POLQA is licensed/commercial where PESQ implementations are free, which is the main reason PESQ remains the de facto research standard despite being technically withdrawn [[ITU-T P.862 status page](#)]. **Keep using PESQ regardless** — the problem statement explicitly names it as a target metric, and it remains overwhelmingly the standard reported in speech-enhancement literature. But if asked, you can note POLQA as the current ITU-designated successor and mention it as a "future work" evaluation addition — this reads as informed rather than caught off guard.

Report structure for your metrics table — break it down exactly the way the problem statement frames the challenge, not just as one averaged number:

Noise category	Input SNR	Enhanced SNR	STOI	PESQ (WB)
Stationary (engine, wind, hum)	...	...	...	...
Non-stationary (helicopter, vehicle)	...	...	...	...
Impulsive (gunshot, artillery, shelling)	...	...	...	...
Held-out generalization (NOISEX-92, unseen speakers)	...	...	...	...

16. Week-by-week build plan (adapt to your actual remaining days)

- Day 1:** Clone Rikorse/DeepFilterNet, run pretrained inference on sample noisy audio to confirm the toolchain works end-to-end. Set up `pesq / pystoi` and confirm you can score a known example against the reference numbers in Section 15.
- Days 2 – 3:** Build the dataset pipeline — DNS Challenge clean speech + noise download/setup, MAD dataset download and class extraction, C3GD gunshot clips, extended SNR-range dynamic mixing script (per Section 3.3). This is the highest-risk, most time-consuming step; start it first, in parallel with everything else if you have more than one person.
- Days 3 – 4:** Fine-tune DeepFilterNet2 on your combined dataset using `df/train.py`. Run a short training job first (few epochs) purely to confirm the training loop, config, and data loader work correctly before committing to a long run.
- Day 5:** Full fine-tuning run (as long as your compute budget allows — GPU hours, not Jetson time, for this stage). In parallel, set up hardware: mic array wiring/pairing, speaker output, Jetson JetPack/CUDA/ONNX Runtime/TensorRT environment.
- Day 6:** Export fine-tuned checkpoint to ONNX (Section 12.2), convert to TensorRT engine, benchmark latency on Jetson. Fork the `deepfilter-rt` streaming reference and adapt it to your exported model.
- Day 7 (go/no-go checkpoint):** Full offline evaluation on your held-out test set — get your PESQ/STOI/SI-SNR table (Section 15) per noise category.

If numbers are close to target, proceed to hardware integration. If far off, this is your decision point to fall back to a smaller scope (e.g., single noise category focus, or DTLN instead of DeepFilterNet2 if latency is the failure mode).

7. **Day 8:** Wire up the live mic → inference → speaker pipeline end-to-end. Add the NLMS residual stage (Section 14) if time allows — it's cheap, do it after the core pipeline is solid, not before.
8. **Day 9:** Live latency measurement (loopback click test), demo rehearsal, raw/enhanced toggle implementation, report writing using the metrics table and citations gathered above.
9. **Day 10 (buffer):** Fix whatever breaks during rehearsal. Something always does — budget for it explicitly rather than treating the schedule as if it won't.

17. What to put in the actual submission report / pitch

Structure that maps directly onto how DRDO framed the problem statement's five deliverables (Section 1) — mirror their own structure back at them:

1. **Background & literature review** — open by citing the DRDO Defence Science Journal paper (Section 10) explicitly; this signals you engaged with DRDO's own institutional framing, not just generic ML literature.
2. **Dataset pipeline** — describe your DNS+MAD+gunshot combination and extended SNR-range mixing strategy; show a sample spectrogram of a synthesized noisy/clean pair per noise category.
3. **Model architecture** — DeepFilterNet2 as primary, with your rationale (ERB-compressed complex-domain masking, proven embedded-real-time track record) and a comparison table against DCCRN/Conv-TasNet if you trained a second model.
4. **Training framework** — your loss function combination (SI-SNR + PMSQE/STOI), and the fact that you evaluated per-noise-category rather than only in aggregate.
5. **Real-time inference engine** — your ONNX/TensorRT pipeline, and your *actual measured* latency number from the live hardware, not a theoretical one.
6. **Live prototype** — mic/headset photos or video, the raw/enhanced toggle, and your live DNSMOS scores if captured during the demo.
7. **Results table** — the per-category PESQ/STOI/SNR breakdown from Section 15, explicitly checked against the stated targets (SNR>15dB,

STOI>0.85, PESQ>2.5), and an honest note on where you fall short (e.g., extreme near-field SNR, per Section 9.3) rather than overclaiming.

8. **Future work** — mention the hybrid NLMS stage, DCCRN+/Distil-DCCRN as scaling paths, and beamforming integration as the natural next steps — this shows depth beyond just "what we managed to finish."

18. Risk register — things likely to go wrong, and what to do about them

Risk	Mitigation
Dataset download/setup eats more time than planned (DNS Challenge is large, MAD requires Figshare access)	Start dataset work on Day 1, in parallel with toolchain setup; have a fallback smaller subset ready
Training doesn't converge / underwhelming PESQ-STOI numbers by the Day 7 checkpoint	Fall back to a narrower scope: focus on 1 – 2 noise categories done well rather than all three done poorly; or fall back to DTLN (smaller, faster to fine-tune)
ONNX/TensorRT export breaks on your fine-tuned checkpoint	Keep the original pretrained ONNX export working as a fallback demo path at all times — never overwrite your last known-working export
Live demo mic picks up unexpected room acoustics / feedback	Rehearse in the actual demo room if possible; have the raw/enhanced toggle ready so a bad live result can still show "here's what it does on our recorded test set" as backup
Judges ask about extreme near-field SNR (point-blank gunfire) and your PESQ isn't near 2.5 there	Have Section 9.3's calibration point ready — state the realistic scope boundary proactively rather than getting caught off guard
Team pivots mid-build to a "better idea"	Lock architecture/dataset decisions by Day 3 and treat Day 7 as the only sanctioned pivot point (fallback within the same architecture family, not a new idea)

19. VAD as a multi-task partner — a real accuracy lever, not just a nice-to-have

Robust voice activity detection (VAD) is genuinely difficult at low SNR, but tightly coupling it to your enhancement model rather than treating it as a separate module gives a measurable boost:

- A framework combining a speech-enhancement decoder and a VAD decoder that **share the same encoder**, optimized jointly via a weighted loss, was shown to give the enhancement network — named VSANet — an improvement over single-task training, specifically attributed to the multi-task setup and an added causal spatial attention block [\[Zhang et al. 2023\]](#).
- A more targeted joint-optimization idea worth stealing directly: **VAD-masked SI-SDR (mSI-SDR)** — instead of training VAD and enhancement as two separate objectives on a shared backbone, the VAD output is used to *mask* the enhancement loss itself during training, coupling the two tasks not just at the shared encoder but at the loss/objective level too, and this was shown to significantly outperform a single-task VAD counterpart while satisfying real-time requirements theoretically [\[Tan et al. 2020\]](#).
- If you don't want to add a full second decoder head, there's a nearly-free option: a multi-task U-Net that estimates an ideal ratio mask (IRM) for enhancement can have its **VAD derived directly from that same IRM output, unsupervised, with no separate VAD model or training required at all** — and this unsupervised-from-IRM VAD performed slightly better than dedicated supervised DNN/LSTM-based VAD baselines specifically under mismatched noise conditions [\[Kim 2020\]](#).

Why this matters for your prototype specifically: a live demo where the model correctly gates output during silence (rather than amplifying residual noise when no one is speaking) is a visibly convincing detail to a judge, and it's also the input signal your NLMS residual stage (Section 14) should ideally use to decide when to adapt aggressively vs. freeze its coefficients (adapt during noise-only segments, freeze/relax during active speech, which is the standard practice in adaptive-filter design to avoid the filter "learning" to cancel speech itself).

20. Refining the NLMS stage: secondary path estimation, and why it's harder than Section 14 implied

Section 14 gave you the basic FxNLMS update rule. One detail that Section 14 glossed over and that will bite you if skipped: the FxNLMS algorithm's derivation **assumes you already know the secondary path** (the transfer function between your control signal / speaker output and where the "error" is actually measured), and this is a real prerequisite, not a formality:

- The standard approach is **offline secondary path modeling** — before deployment, play a known white-noise (or similar) probe signal through your speaker, record it at the error mic, and fit an adaptive filter to model that transfer function; this pre-identified secondary path is then held fixed and used to "filter" your reference signal in the FxNLMS update [[A New FXLMS Algorithm With Offline and Online Secondary-Path Modeling, Zhao et al. 2017](#)] — this exact offline+online hybrid approach was validated on a real transformer noise-control deployment, achieving **8 – 15 dB noise reduction and an 84 – 97% decrease in average sound energy density**, giving you a real precedent for how much a well-implemented FxLMS-family system can achieve even outside your DNN stage entirely.
- If your headset/speaker-to-mic geometry is fixed and known ahead of time (which it likely is, for a prototype demo), a one-time offline calibration pass the morning of your demo is sufficient — you don't need online/adaptive secondary-path re-estimation, which is what most of the advanced ANC literature (Section 20's remaining citations) is actually solving for (time-varying secondary paths in moving vehicles, rotating machinery, etc. — not your use case).
- **If you want to skip secondary-path modeling entirely** (simplest possible path, reasonable for a hackathon timeline): genetic-algorithm-based and other secondary-path-free ANC formulations exist specifically to eliminate this requirement, reformulating ANC as a direct optimization problem instead of a gradient-based one requiring a pre-known transfer function [[Zhou et al. 2023](#)] — worth knowing this exists as a fallback if secondary-path calibration proves finicky under time pressure, though it's a more novel/less battle-tested path than standard offline-modeled FxNLMS.

Practical recommendation for your timeline: do a **5-minute offline secondary-path calibration** (play white noise through your output speaker, record via error mic, fit a short FIR filter — a few dozen taps is enough for a simple headset geometry) as a fixed pre-step before your live demo, rather than attempting online secondary-path adaptation. This is the standard, low-risk choice and matches what most working FxNLMS deployments actually do in practice.

21. TensorRT INT8 quantization — the exact workflow, if you need it

Section 5.2 covered FP16 mixed precision, which is likely sufficient given your compute headroom on Jetson AGX Orin. If you do want to push to INT8 (e.g., targeting a smaller/cheaper Jetson variant, or simply to demonstrate depth of optimization work in your report), here is the actual mechanism, not just the concept:

Post-Training Quantization (PTQ) — the practical default, lower engineering cost than QAT:

1. Export your fine-tuned PyTorch model to ONNX (Section 12.2 — you already have this step).
2. TensorRT's built-in calibration process **measures the distribution of activations within each activation tensor as the network runs on representative input data**, then uses that measured distribution to estimate INT8 scale factors for each tensor [\[NVIDIA TensorRT Developer Guide\]](#) — practically, this means you feed the calibrator a batch of representative *noisy audio feature* samples from your own validation set, not random data or (for instance) an unrelated public calibration set.
3. Set the calibrator on your TensorRT builder config (`config.int8_calibrator = calibrator`), enable the INT8 build flag (and FP16 alongside it if you want a mixed-precision "hybrid" engine, which TensorRT explicitly supports and is often the safer choice — leave numerically sensitive layers like your first/last convolutions or the GRU state layers in FP16 while quantizing the bulk of the convolutional stack to INT8) [\[NVIDIA/TensorRT GitHub issue #3978, worked example\]](#).
4. Build the engine, save the calibration cache (so you don't have to re-run calibration on every rebuild), and benchmark against your FP16 baseline — only keep the INT8 engine if it doesn't measurably degrade your PESQ/STOI numbers on your held-out test set.

How much calibration data you actually need: precedent from a comparable real-time edge deployment (vehicle speed estimation via YOLOv6/TensorRT) used a calibration set of **32 batches of size 32** (roughly 1,000 samples) drawn from training data [\[Efficient Vision-based Vehicle Speed Estimation\]](#) — a few hundred to a thousand representative noisy-speech feature windows from your validation set is a reasonable target, not your entire dataset.

Honest recommendation: given Jetson AGX Orin's massive compute headroom relative to a model in the DeepFilterNet2 parameter-count class,

INT8 is very likely unnecessary for your actual deployment — FP16 mixed precision (Section 5.2) should already give you comfortable real-time margin. Do the INT8 pass mainly if you want the extra "we pushed optimization further than needed for headroom, to show it generalizes to smaller future hardware" line in your report, not because you need it to hit your latency target.

22. Physical ruggedization — what a "real" defence prototype needs to at least acknowledge

Your hackathon prototype doesn't need to pass military qualification testing, but **explicitly acknowledging the ruggedization requirements your design would need for actual fielding** is a strong signal of understanding the deployment context beyond the lab bench, and DRDO evaluators will be attuned to this gap if left unaddressed.

The relevant standard is **MIL-STD-810** (currently Revision H, Change 1), which is a *framework of ~29 individual test methods* rather than a single pass/fail certification — a manufacturer selects and "tailors" the relevant subset of tests to the equipment's actual intended environment [[MIL-STD-810 overview, Crystal Group](#)]. The methods most relevant to your ANC prototype specifically:

- **Vibration and shock** — relevant since the problem statement explicitly targets vehicle/aircraft-mounted use cases; vibration sources are tracked/wheeled vehicle operation, aircraft, and equipment handling [[MIL-STD-810 Testing, Crystal Group](#)].
- **Temperature extremes** — most rugged defence electronics are tested/rated for roughly **-20°C to 60°C operating range**, tailorable to more extreme conditions depending on deployment theatre [[MIL-STD-810G/H, Estone Technology](#)].
- **Sand and dust, and humidity** — directly relevant to your microphone housing design; this is typically paired with an **IP rating** (Ingress Protection) for the physical enclosure — note that IP ratings and MIL-STD-810 are complementary, not interchangeable: IP measures dust/water resistance specifically, while MIL-STD-810 covers a much broader stress envelope including vibration and thermal shock that an IP-rated-only enclosure may not survive [[MIL-STD-810G/H, Estone Technology](#)].
- **EMI/EMC** — for a system that will sit near tactical radios, mention **MIL-STD-461** (electromagnetic compatibility) as a companion standard your

final design would need to address, particularly relevant since RF interference from nearby radio transmitters is a realistic operational concern for a comms-adjacent device [[MIL-STD-810: Environmental Testing, Betalight Tactical](#)].

What to actually put in your report: a short "Deployment Readiness" subsection stating that your current prototype is at proof-of-concept fidelity (algorithm + real-time hardware demo), and that a fielded version would require MIL-STD-810H vibration/shock/temperature qualification and MIL-STD-461 EMC testing for the enclosure and mic array specifically — this maps naturally onto standard **Technology Readiness Level (TRL)** language, which DRDO/iDEX evaluators use explicitly: your prototype is realistically a **TRL 3–4** (proof of concept validated in a lab/relevant environment), and stating this honestly, along with a clear TRL-5-and-beyond roadmap (ruggedization, field trials), reads as more credible than implying a finished product [[DRDO TDE Scheme TRL criteria](#)].

23. Program context — what track this problem statement likely sits in, and what that implies for your submission

Based on the problem statement's format (numbered PS ID, "Organization: DRDO", "Department: Department of Defence Production/iDEX", "Category: Hardware"), this is very likely structured as an **iDEX (Innovations for Defence Excellence)** challenge, quite possibly under the **DISC (Defence India Startup Challenge)** or a similar iDEX track. Useful context if you weren't already aware of the broader program:

- iDEX runs milestone-based grant funding (the **SPARK** — Support for Prototype & Research Kickstart — scheme) of **up to ₹1.5 crore**, disbursed in tranches tied to prototype development milestones, specifically for turning a winning problem-statement response into a real, fieldable prototype — meaning your Techstorm/hackathon-stage submission may realistically be a stepping stone toward a much longer funded development track, not a one-shot event [[iDEX DISC 14 overview](#)].
- Some DRDO innovation tracks (e.g., Dare to Dream) use a **blind evaluation process** — proposals submitted with no identity markers (name, institution, logo) embedded in the document itself — worth double-checking your specific competition's submission portal rules before you finalize your report, since accidentally embedding your college/team

branding inside a PDF that's supposed to be anonymized is a real, avoidable disqualification risk in DRDO-style evaluations generally [[DRDO Dare to Dream overview](#)].

- Evaluation for these programs typically emphasizes **feasibility, technical viability, and a credible pathway to a fieldable product** — not just an isolated demo — which is exactly why Sections 17 (report structure), 22 (ruggedization/TRL framing), and this section together matter: showing you understand what happens *after* the demo, not just that the demo works, is a distinguishing factor DRDO reviewers explicitly look for [[TDE Scheme evaluation criteria](#)].

Actionable takeaway: add one slide/paragraph to your pitch explicitly addressing "path to TRL 6+" (field-representative environment testing, ruggedization, integration with an actual comms/PPE platform) — even a few honest sentences here meaningfully differentiates your submission from ones that stop at "here's our demo."

24. Completeness check — what this document now covers, end to end

At this point the document spans the full stack DRDO's problem statement asks for, cross-referenced against real implementation and literature at every layer:

- **Problem framing & DRDO's own institutional view** (Sections 1, 10, 23)
- **Model architecture selection & justification** (Section 2, with peer-reviewed benchmark numbers in Section 9)
- **Dataset construction, including defence-specific sources** (Section 3)
- **Training: features, loss functions, multi-task VAD integration** (Sections 4, 19)
- **Real-time embedded deployment: ONNX, TensorRT, FP16/INT8** (Sections 5, 21)
- **Hybrid classical+DNN ANC stage, including the secondary-path prerequisite most guides skip** (Sections 6, 14, 20)
- **Physical integration: mic hardware, beamforming architecture pattern** (Sections 7, 11.1, 13)
- **Exact repos, commands, and streaming-inference references to fork rather than reinvent** (Section 12)

- **Evaluation methodology, including the no-reference-metric gap for live demos** (Sections 11.2, 15)
- **Build timeline, submission structure, and risk mitigation** (Sections 16, 17, 18)
- **Ruggedization and program/funding context for the "beyond the demo" story** (Sections 22, 23)

The remaining unknowns are ones only you can resolve by building and testing: your actual measured latency numbers, your actual PESQ/STOI/SNR results per noise category, and how your specific hardware/mic geometry behaves — everything upstream of "run the experiment and see" is now covered with a citable, checkable source. If a genuinely new question comes up while building (a specific bug, an unexpected result, a hardware quirk), bring it back and it can be researched the same way.

25. Synthesis — three additions worth building into your actual prototype now

A second independent research pass (run by the user through another tool) converged on much of the same material independently — a good cross-validation signal — but also surfaced three ideas cheap enough to fold into your existing 10-day build plan without expanding scope. These are additions, not replacements, for Sections 2 – 18 above.

25.1 Frame your system as a controller, not a black-box denoiser

Instead of describing your pipeline as "AI takes noisy audio in, enhanced audio comes out," describe it as **an AI-driven controller that orchestrates classical DSP**:

```

Speech confidence + noise-state estimate + impulse flag
      ↓
    Adaptive Controller
(suppression strength, NLMS step size, freeze/release)
      ↓
AI Enhancement (DeepFilterNet2) + NLMS residual stage
      ↓
    Protected output

```

You do **not** need four separately trained neural networks to earn this framing. A lightweight, cheap-to-build version:

- **Speech confidence:** derive it directly from your enhancement model's own output mask/gain, unsupervised — no separate VAD model needed (Section 19's IRM-derived VAD trick).
- **Impulse flag:** a simple runtime feature detector — crest factor, short-term energy ratio, and spectral flux computed frame-by-frame, thresholded — not a trained classifier. This is classical DSP, implementable in an afternoon, and it's exactly what should gate your NLMS adaptation (freeze coefficients during a detected impulse per Section 20, rather than letting the filter try to "learn" a gunshot as if it were stationary noise).
- **Noise-state (stationary/dynamic/impulsive):** for v1, this can literally be the impulse flag plus a short-term vs. long-term energy variance ratio — you don't need Module A as a trained classifier to tell a credible "the system adapts its behavior based on noise state" story.

This reframing costs you almost nothing in implementation time and meaningfully changes how your report and demo narrative land — "our AI controls a signal-processing system" is a stronger claim than "we trained a CNN," and it's true even at this lightweight scope.

25.2 Reference microphone placement — treat it as a 30-minute experiment, not a given

Before finalizing your mic rig, test 2 – 3 reference mic positions with your actual headset/enclosure geometry (e.g., near-ear vs. rear-of-earcup vs. boom) and measure Δ SNR and speech leakage into the reference channel for each, using a fixed noise source and fixed speaker position. Report the winning configuration and the actual dB delta you measured — a small, concrete experimental result like this is disproportionately convincing in a demo/report relative to its cost, and directly pre-empts a judge asking "why did you put the reference mic there?"

25.3 State your latency budget before you have final numbers

Add a table like this to your report early, even before your hardware benchmarking is finished — filling it in with real measurements later is easy, and stating the budget upfront reads as engineering rigor regardless of final numbers:

Stage	Target budget
ADC / input buffering	2 – 5 ms

Stage	Target budget
STFT / feature extraction	1 – 3 ms
AI inference (DeepFilterNet2, per Section 12.3's measured ~1.7ms/10ms frame)	3 – 10 ms
NLMS post-processing	<1 ms
DAC / output	2 – 5 ms
Safety margin	remainder
Total target	<30 ms

This also gives you a principled way to answer "why 10ms frames and not 32ms STFT windows" if asked — you chose the frame size to fit a latency budget you set deliberately, not by default.

26. Phase 2 roadmap — explicitly deferred, not forgotten

The second research pass proposed a substantially larger system than your competition timeline supports: four independently-trained AI modules (noise-state classifier, speech-confidence network, enhancement network, adaptive controller), physical loudspeaker-based ANC with secondary-path delay compensation, 2 – 3 mic spatial/beamforming processing, online domain adaptation, and dedicated model compression research. Each idea is individually legitimate and several converge with peer-reviewed literature already cited in this document — but attempting all of it in parallel is the classic failure mode of a strong-but-overscoped student project: five half-finished subsystems instead of one that works convincingly. Keep this as an explicit **"Future Work" section** in your report — it demonstrates depth and a real roadmap beyond the demo (which Section 23 already establishes matters for how DRDO evaluators read a submission) without risking your actual deliverable:

- **v2 — Multi-mic spatial processing:** 2 – 3 mic array, MVDR/GSC beamforming ahead of the DNN (Section 11.1's already-cited beamform-then-postfilter pattern), only after the single-reference-mic v1 is solid and its limits are actually measured, not assumed.

- **v3 — True physical ANC as a residual layer:** loudspeaker-driven anti-noise specifically for low-frequency structured noise (engine rumble, rotor fundamentals) where the secondary acoustic path is predictable and controlled — explicitly scoped to low-frequency/narrowband noise, not broadband cancellation, per the second pass's own correct caution in its §18-19.
- **v4 — Trained noise-state and speech-confidence modules:** replace the lightweight heuristic controller (Section 25.1) with actual trained classifiers once you have the labeled runtime telemetry from v1 to train them on.
- **v5 — Model compression as its own research thread:** structured pruning, integer quantization, and state-update skipping for a target smaller than Jetson-class hardware (the second pass cites a claimed 11.9x size / 2.9x op reduction via this combination for a "TinyLSTM"-style approach — worth verifying against the original source before citing it yourself, since it wasn't independently checked here the way the LiSenNet numbers above were).
- **v6 — Online runtime adaptation:** adjusting normalization statistics, suppression strength, and gain based on the live acoustic environment, without retraining the network itself — a reasonable production-hardening step, explicitly out of scope for a first prototype.

One explicit caution on sourcing: the second research pass contained zero citations, unlike every claim in Sections 1 – 24 above, which link to a checkable paper or primary source. One specific, checkable number from it (LiSenNet: 37k parameters, 56M MAC/s) was independently verified against the original paper and confirmed accurate — a reasonable, though not exhaustive, signal that the rest is likely well-grounded. Treat any other specific figures from that pass (e.g., "0.70G MACs/s, RTF 0.012" for an unnamed model) as directional rather than confirmed until you verify them the same way, especially before putting them in a written report DRDO evaluators will read closely.

27. Evidence Framework — how to treat every claim in this document (including its own)

A cross-check against a third, more disciplined synthesis pass surfaced a genuinely useful practice this document should have enforced from the start: **label every claim by its evidence status**, and never let a literature number quietly become a promise about your own hardware.

Use three labels going forward, for anything you write in your own report too:

Label	Meaning	Example from this document
VERIFIED RESEARCH	A specific, checkable number from a real paper/source, under <i>its</i> stated conditions	DeepFilterNet2's reported real-time factor of 0.04 on a notebook Core-i5; LiSenNet's 37k parameters / 56M MAC/s (Section 26, independently confirmed against the original paper)
ENGINEERING TARGET	A goal you're setting for your own system, not a literature claim	The $\leq 20 - 30$ ms end-to-end latency budget (Section 25.3) — this is <i>your</i> target, to be measured, not something guaranteed by any cited paper
HYPOTHESIS	A plausible claim this document has been treating as true but that you haven't tested yet	"AI + NLMS outperforms AI alone" — stated as fact in Sections 6 and 14, but is actually untested on your data until you run the ablation in Section 30

Self-audit of this document's own weakest point: Sections 6 and 14 present the DNN+NLMS hybrid stage as though it's self-evidently beneficial. It isn't — that's a hypothesis, and the honest instruction is the same one Section 30 below states explicitly: **if your ablation shows NLMS adds no measurable improvement, cut it from your core claim rather than keeping it because it was planned.** A hybrid architecture that doesn't earn its complexity through measured evidence is a weaker report than a simpler one that's honestly benchmarked.

Apply this same three-way labeling discipline to your own report's claims before submission — it's one of the cheapest credibility upgrades available to you, and DRDO/iDEX evaluators reading dozens of submissions will notice a team that clearly separates "we measured this" from "we expect this" from "the paper said this."

28. Scope Freeze — what you are actually building first

This section is the authoritative, final version of the scope boundary Section 25/26 already began drawing — restated here precisely, so there's one unambiguous answer to "what are we building" that doesn't drift as the research keeps expanding.

In scope for the first build:

- Primary microphone + reference microphone (Section 13's hardware options)
- 16kHz speech-communication audio for the first implementation (upgrade sample rate later if margin allows)
- Streaming STFT pipeline with causal or tightly-bounded lookahead (Section 12.3's streaming reference)
- **DeepFilterNet2** (or a compact DeepFilterNet-derived model) as the first AI benchmark; **compact CRN as the fallback** if DeepFilterNet2 integration proves harder than expected
- A simple **NLMS residual stage** (Section 14/20) — treated as an experimentally testable component per Section 27, not an assumed benefit
- A **rule-based controller** (Section 25.1) driven by speech/VAD confidence, crest factor, energy, and reference-coherence checks — explicitly *not* separately trained neural networks
- Embedded streaming prototype with measured latency, CPU/RAM, and audio stability
- A controlled acoustic test rig with repeatable benchmark recordings (Section 29 below)
- The mandatory comparison: noisy vs. AI-only vs. AI+NLMS (Section 30)

Explicitly excluded from the first build (all captured in Section 26's Phase 2 roadmap, not discarded):

- Separately trained noise-state, speech-confidence, and impulse-detection *networks* (use the cheap heuristic versions instead — Section 25.1)
- Neural beamforming and multi-mic arrays beyond primary+reference
- Online neural domain adaptation
- Diffusion/GAN-based enhancement
- Full broadband physical ANC (loudspeaker-driven anti-noise) — Section 26's v3 already scopes this correctly as a low-frequency-only future extension, not a v1 feature
- Custom accelerator/PCB design

If you find yourself tempted to add anything from the excluded list before the frozen scope is working end-to-end, that's the signal to write it into Section 26 instead of your build queue.

29. Baseline experiment matrix and controlled demo rig

Before the "impressive" hybrid system is worth anything in your report, you need it measured against simpler baselines — otherwise you can't actually claim the added complexity helped.

Baselines to run, in order of increasing complexity:

1. Unprocessed noisy speech (the floor)
2. Classical spectral subtraction (a 1970s-era baseline — cheap to implement, useful as the "why not just do this" comparison)
3. NLMS alone (no AI)
4. AI-only (DeepFilterNet2, no NLMS stage)
5. AI + NLMS (your full hybrid)
6. AI + NLMS + rule-based controller (your complete v1 system)

Test conditions matrix — run every baseline above against each of: engine/stationary noise, rotor/siren non-stationary noise, wind, impulsive events, mixed noise, and a held-out unseen-noise set (NOISEX-92, per Section 3.3). This is the same matrix as Section 15's per-category metrics table, but now crossed against every baseline tier rather than just your final system — it's what actually lets you claim "the hybrid architecture earns its complexity," rather than just asserting it.

Controlled physical demo rig: build a repeatable acoustic test zone with fixed source geometry — separate, fixed positions for the speech source, the noise source, and the headset/mic rig, documented and reproducible run to run. Your live demo sequence should walk through a deliberate escalation: raw speech → stationary engine noise → non-stationary rotor/siren noise → an impulsive event → mixed noise — while displaying a live waveform/spectrogram, your benchmark SNR/STOI/PESQ numbers, and live latency/CPU usage on screen. The physical headset and processing unit should visibly be the centerpiece of the demo; any software dashboard should support that hardware demonstration, not substitute for it.

30. Mandatory ablation — the actual table to fill in

This is the single most important table in your final report, because it's the only thing that actually proves the hybrid architecture (rather than just DeepFilterNet2 alone) was worth building:

System	Engine	Rotor/Siren	Wind	Impulse	Mixed	Unseen
Noisy (unprocessed)						
Spectral subtraction baseline						
NLMS only						
AI only (DeepFilterNet2)						
AI + NLMS						
AI + NLMS + controller						

Fill each cell with your PESQ (or the full SNR/STOI/PESQ triple if space allows). **The rule to actually follow, not just state:** if a row doesn't measurably beat the row above it in the conditions it's meant to help with, don't keep claiming it as a contribution in your write-up — report it honestly as a negative result instead ("we found NLMS added no measurable benefit over AI-only for non-stationary noise, and hypothesize this is because X") — a well-reasoned negative result reads as more credible than an unsupported positive claim, especially to technically literate DRDO evaluators.

31. Fail-safe / runtime health design — the mission-critical-systems layer

A defence communication device should degrade gracefully, never fail silently or become unusable because a neural model hiccups. This is cheap to implement (mostly conditional logic wrapping your existing pipeline) and is one of the strongest "we understand this is for defence, not just a class project" signals you can put in front of DRDO evaluators.

State machine to implement:

AI healthy, confidence high adaptation	→ AI enhancement + normal NLMS
AI healthy, confidence low (reduced suppression aggressiveness)	→ conservative enhancement
AI inference failure / timeout processing	→ fall back to NLMS/DSP-only
Hardware/audio fault detected mic audio through unmodified)	→ transparent bypass (pass raw mic audio through unmodified)
Input clipping detected	→ engage limiter, log the event, flag for post-session review
Reference mic judged unreliable adaptation (don't let a bad reference signal corrupt the filter)	→ reduce or freeze NLMS

What "confidence" and "reliability" mean concretely, without extra trained models: confidence can be the enhancement model's own output-mask statistics (Section 25.1's free VAD trick extended); reference-mic reliability can be a simple coherence/correlation check between primary and reference channels — low coherence means the reference mic isn't capturing what you think it's capturing (speech leakage, mismatch, or a physically dislodged mic), and is exactly the condition Section 5 warned about where a "reference" mic can actively damage speech if trusted blindly.

Runtime monitor to log and display during your demo: latency (per-frame and worst-case), CPU/RAM utilization, clipping events, current controller mode, and confidence/coherence scores — this is what turns "here's our system" into "here's our system, and here's proof it knows its own limits," which is a substantively stronger claim for a defence-context evaluator to see live.

32. Acceptance criteria — the honest bar for calling v1 "done"

Before you call the first build finished, it should be able to demonstrate all of the following, not just some:

- Suppression performance across stationary, non-stationary, and impulsive scenarios specifically (not just an aggregate number)
- SNR > 15dB, STOI > 0.85, PESQ > 2.5 on clearly defined, stated test conditions — and explicitly stated input SNR alongside output SNR for every headline number, never output-only

- **Real-time streaming operation**, demonstrated live — not an offline batch-processed recording presented as if it were real-time
- Measured end-to-end latency, CPU/RAM, and model size, reported alongside your quality numbers, not separately
- A live microphone → processing → headset demonstration, not just a software plot
- Comparison against the baseline tiers in Section 29/30, with honest reporting of unseen-noise generalization results, including where they fall short

33. Extended roadmap — 30/60/90 days, for context beyond the immediate competition deadline

Section 16 already gave you a tight day-by-day plan for an imminent hackathon deadline. If this problem statement continues into a longer iDEX/SPARK-funded track after the initial competition (a real possibility per Section 23's program context), here's how the same work extends:

- **Days 1 – 30:** DSP fundamentals; implement STFT/iSTFT from scratch to understand it, not just call a library; build spectral-subtraction and NLMS baselines; build the dataset generator (Section 3); establish the SNR/STOI/PESQ evaluation harness (Section 15); get one streaming audio path working end-to-end, even a bad one — proving the pipeline plumbing works is the actual Day-30 milestone, not model quality yet.
 - **Days 31 – 60:** train the compact AI baseline (DeepFilterNet2 fine-tune, Section 12); benchmark candidate architectures against each other (Section 2's DCCRN comparison); add dedicated impulsive-noise training; build the AI+NLMS hybrid and start the ablation matrix (Section 30); establish unseen-noise testing discipline early rather than only at the end.
 - **Days 61 – 90:** deploy on real edge hardware (Section 5); optimize via ONNX/TensorRT/quantization (Sections 12.2, 21); integrate the actual I2S microphones and headset hardware (Section 13); measure latency/RAM/CPU/power comprehensively; build the enclosure and the controlled acoustic test rig (Section 29); move from "it works on my laptop" to "it works as a physical demonstrable object."
-

34. Team learning checklist — what everyone building this should actually know

A practical map from "topic" to "why it matters for this specific prototype," useful for splitting study/research load across a team:

Area	Must know	Why it matters here
Digital audio	Sampling, PCM, dB scales, ADC/DAC, dynamic range	Governs what your front end can and can't capture — Section 13/14's clipping and headroom issues live here
Signal processing	FFT, STFT, windows, filters, phase	The literal core of every enhancement model in Section 2
Adaptive filtering	LMS, NLMS, FxNLMS, convergence, step size	Sections 14/20 — your residual/hybrid stage
Speech processing	Speech spectrum, voiced/unvoiced sounds, harmonics	Needed to reason about why your model preserves or damages speech
Deep learning	CNN/RNN/LSTM/CRN, masks vs. complex filtering, causal inference	Section 2's architecture landscape
Data engineering	SNR mixing, RIR convolution, augmentation, train/test leakage prevention	Section 3 — the step every team underestimates
Evaluation	SNR, SI-SNR, STOI, PESQ, WER, DNSMOS	Sections 9, 11.2, 15
Real-time systems	Buffering, causality, latency budgeting, DMA	Section 25.3's latency table exists because of this
Embedded AI	ONNX, TensorRT, quantization, profiling	Sections 5, 12.2, 21
Hardware	MEMS mics, I2S, DAC/codec, power, enclosure	Section 13

35. Military voice codec integration — a gap every civilian-dataset-trained model misses

This is a genuinely important, previously-uncovered gap: your enhanced speech almost certainly won't be the final signal transmitted. In a real tactical radio chain, it gets encoded by a military vocoder afterward — and that downstream codec has requirements your model needs to be compatible with, not just "sound good" in isolation.

The relevant standard is **MELPe (Mixed-Excitation Linear Prediction, enhanced)**, standardized as **STANAG 4591** (NATO) and **MIL-STD-3005** (US DoD) — the current NATO/DoD standard narrowband vocoder for secure tactical voice, operating at 2400/1200/600 bps [[STANAG 4591 overview, rapidm.com](#)]. Concrete constraints worth knowing:

- **MELPe requires a nominal analog bandwidth of 100Hz – 3800Hz** and 16-bit linear A/D conversion at 8kHz [[STANAG 4591, rapidm.com](#)] — meaning the actual downstream radio chain your enhanced speech would feed into is **narrowband, not the 48kHz fullband** DeepFilterNet3 natively operates on. This is a real design decision point: either (a) explicitly position your prototype as enhancing the *microphone-to-headset* link (before any radio encoding), which is squarely what the problem statement's "headset/communication unit" language supports, and 48kHz fullband processing is entirely appropriate there; or (b) if you want to demonstrate radio-chain compatibility explicitly, add an 8kHz narrowband evaluation condition to your test matrix, since that's what a MELPe-based radio would actually see.
- **MELPe itself already includes a noise pre-processor (NPP)** — it adaptively estimates the noise spectrum and attenuates noise components before encoding [[TSVCIS/MELPe, vocal.com](#)]. This is directly relevant framing for your report: your prototype doesn't compete with an empty gap in the pipeline, it upgrades a stage (MELPe's NPP) that DRDO's own comms infrastructure already knows is inadequate for the "highly dynamic and non-linear noise environments" your problem statement calls out — a strong, specific way to state your system's value proposition rather than a generic "AI is better than classical DSP" claim.
- **The Indian defence forces' own tactical radio ecosystem** (as with most NATO-interoperable forces) is very likely to use MELPe-class narrowband vocoders in at least some radio links, even if higher-bandwidth digital links exist elsewhere — worth a single sentence in your report acknowledging this downstream constraint exists, since it signals you understand where

your prototype sits in a larger communication chain rather than treating it as a standalone endpoint.

Practical recommendation: keep your primary enhancement pipeline full-band (48kHz, DeepFilterNet2/3) for the headset-level demo, since that's the strongest showcase of your AI system's capability — but add one evaluation row to your metrics table (Section 15/30) at 8kHz/narrowband to show you've considered radio-chain compatibility. Mentioning MELPe by name in your report, even briefly, is a low-cost signal of domain depth that most student teams won't have.

36. Hearing safety standards — the "why this matters" context your report's motivation section is missing

Section 11.4 already cited human-factors studies on intelligibility degradation; this adds the actual regulatory hearing-safety standard, which is a more concrete and citable framing for your report's introduction than generic "noise is bad" motivation.

MIL-STD-1474E is the current US DoD design-criteria standard for noise limits in military materiel, and its two headline numbers are worth quoting directly: it requires **steady-state noise levels below 85 dBA and impulsive peak-pressure levels below 140 dB(P)** at the ear (protected or unprotected) at occupied locations during normal operation [[Noise Limits for Warfighting, AIHA](#)]. Critically, most weapons systems **exceed this by a wide margin** — few if any weapons produce impulse levels below 150 dB(P), and levels can reach 180+ dB(P) [[MIL-STD-1474E, ResearchGate](#)]. This is exactly the gap your problem statement is implicitly responding to: hearing protection and clear communication are being asked to coexist in an environment where the noise sources are, by design specification, already known to exceed safe limits.

Two more specifics worth citing directly if you want a rigorous motivation section:

- The DoD's broader Hearing Conservation Program (DoDI 6055.12) sets an **85 dB(A) action level as an 8-hour time-weighted average**, using a stricter 3dB exchange rate (matching EU practice) rather than OSHA's more permissive 5dB rate [[Military Noise Exposure Limits, hearingprotect.com](#)].
- For impulse noise specifically, the U.S. Army's **AHAH (Auditory Hazard Assessment Algorithm for Humans)** model computes **Auditory Risk Units (ARUs)** from a blast waveform, where values over 500 ARUs for a 24-hour exposure are associated with likely permanent hearing loss [[AHAH/](#)

[MIL-STD-1474E, ResearchGate](#) — this is the actual computational model DRDO-adjacent hearing-protection research would reference, and namechecking it (even without implementing it) shows you've looked past "noise is loud" into the actual engineering literature the problem statement's own domain uses.

One line worth adding to your report's introduction: your system doesn't just improve speech quality — it operates in exactly the acoustic regime (85dB+ steady-state, 140dB+ impulsive peaks) that military design standards already acknowledge as hazardous and communication-degrading by specification, which is a more precise and citable framing than a generic appeal to "battlefield noise is a problem."

37. Jetson power management — the practical knob you'll actually touch during hardware bring-up

Section 5.1 established that compute headroom isn't your bottleneck on Jetson AGX Orin. What wasn't covered: the actual power-mode configuration you'll need to set correctly during hardware bring-up, since getting this wrong is a common, avoidable source of "why is my latency inconsistent" confusion during a live demo.

Jetson AGX Orin exposes discrete power profiles via `nvpmode.l` — for the 64GB developer kit, sample modes are provided for **15W, 30W, and 50W**, plus a MAXN mode that removes the power ceiling entirely (up to 275 TOPS, drawing up to 60W) [\[AGX Orin power consumption, NVIDIA forums\]](#). Practical commands:

```
sudo nvpmode.l -m 0      # switch to a specific power mode (0 =
MAXN on most Orin configs)
sudo jetson_clocks       # lock CPU/GPU/EMC clocks to the max
frequency allowed by the current power mode
sudo tegrastats          # live power/thermal/utilization monitor –
use this during your latency benchmarking
```

`jetson_clocks` matters specifically because **without it, the Jetson uses dynamic frequency scaling and defaults to conservative clock speeds** (e.g., GPU sitting around 600MHz) to preserve thermal/power margins — meaning your first "unoptimized" latency benchmark may look significantly worse than what your hardware can actually do, purely because of a default power setting, not anything about your model [\[Enabling Maximum Performance Mode, dev.to\]](#). Given your model is small relative to Orin's headroom (Section

5.1), you likely don't need MAXN/60W for real-time performance — but you should **explicitly state and report which power mode you benchmarked under** (e.g., "measured under nvpmode MODE_30W with jetson_clocks locked"), since an unstated power mode makes your latency number unreproducible and is exactly the kind of detail a technically rigorous evaluator will ask about.

A genuinely relevant tradeoff for a portable defence prototype: if your final demo unit needs to run on battery rather than wall power, the 15W mode is the realistic operating point for a headset-form-factor device, not MAXN — worth explicitly benchmarking your model's latency *at 15W specifically*, since that's the constraint a real fielded version would actually operate under, and reporting "we hit our latency target even in the lowest 15W power mode" is a substantially stronger claim than reporting a MAXN-mode number that a battery-powered fielded unit could never sustain.

38. Real-time audio I/O software stack — the plumbing between your mic and your model

Section 12.3 covered the model-side streaming inference. This covers the other half — the actual audio input/output layer that feeds it, which is a common, under-discussed source of glitches, dropouts, and inconsistent latency in student real-time-audio projects.

On Linux (including Jetson's Ubuntu-based JetPack OS), the practical stack is: ALSA (the kernel-level audio driver layer) as the foundation, with **PortAudio** (via Python's `sounddevice` library) as the cross-platform abstraction on top of it — this is the standard, well-documented combination for low-latency Python audio I/O [[PortAudio Linux build docs](#)]. Critically, **`PaAlsa_EnableRealtimeScheduling` needs to be enabled** so ALSA runs at a high scheduling priority and isn't preempted by ordinary processes — without this, low-latency audio playback becomes irregular with frequent dropouts, which is exactly the kind of intermittent glitch that's hard to diagnose live at a demo table if you haven't already ruled it out during development [[PortAudio Linux build docs](#)].

For genuinely demanding low-latency work (smaller than the ~10ms frame budget you're targeting), **JACK** (via `jackd`) is the more specialized option, commonly run with a real-time scheduling priority flag and a small frames/period buffer setting — a documented reference configuration for a Raspberry-Pi-class embedded Linux device (directly analogous to your Jetson target) uses **a 128-frame period with 3 periods per buffer**, prioritized at

real-time scheduling priority 70 [[Raspberry Pi realtime audio, wiki.linuxaudio.org](#)] — a reasonable starting point for your own buffer tuning rather than guessing.

One architectural detail worth building in from day one, not retrofitting later: implement your audio callback as a pure ring-buffer read/write with **no processing inside the real-time audio callback itself** — hand data off to a separate processing thread and let the callback stay minimal and deterministic. This is the standard pattern used by low-latency Python audio libraries specifically because Python's GIL and garbage collector can introduce unpredictable pauses inside a callback, and an audio callback that blocks even occasionally causes audible clicks/dropouts that are very hard to debug after the fact [[python-rtmixer documentation](#)]. Practically: your mic-in callback should just push raw frames into a ring buffer; your inference loop (running in a separate thread) pulls from that buffer, runs the model, and pushes results into a second ring buffer that your output callback reads from — never call your ONNX Runtime/TensorRT inference directly inside the audio driver's callback function.

Key sources for further reading

- DeepFilterNet2 paper: arXiv:2205.05474
- DeepFilterNet official repo: github.com/rikorose/deepfilternet
- Real-time ONNX streaming reference: github.com/shimondoodkin/deepfilter-rt
- DCCRN paper (Interspeech 2020 DNS Challenge winner); DCCRN+: Lv et al. 2021; Distil-DCCRN: Han et al. 2024
- FullSubNet: arXiv:2010.15508
- Conv-TasNet low-latency variant: arXiv:2404.19375, arXiv:2204.09911
- DNS Challenge (Interspeech 2020/2021): arXiv:2005.13981
- MAD (Military Audio Dataset): Nature Scientific Data, 2024 — [nature.com/articles/s41597-024-03511-w](https://www.nature.com/articles/s41597-024-03511-w)
- C3GD Gunshot Dataset: arXiv:2606.18135
- SFANC-FxNLMS hybrid ANC: arXiv:2208.08082
- FxNLMS step-size analysis: arXiv:2601.16382
- PMSQE perceptual loss: Martín-Doñas et al.
- DNSMOS / DNSMOS P.835: Reddy et al. 2020, 2021
- Jetson AGX Orin 64GB datasheet: NVIDIA / developer.nvidia.com

- reSpeaker XVF3800 mic array: seeed.cc / wiki.seeedstudio.com
- DRDO Defence Science Journal (this exact problem): DOI 10.14429/dsj.21095, Vol. 76 No. 3, May 2026
- Evaluation libraries: pesq (ludlows/python-pesq), pystoi ([mpariente](https://github.com/parizot/pystoi)), pysepm ([schmiph2](https://github.com/schmiph2))
- VAD multi-task speech enhancement: Zhang et al. 2023 (VSANet), Tan et al. 2020 (mSI-SDR)
- FxLMS/secondary path modeling: Zhao et al. 2017; Zhou et al. 2023 (secondary-path-free GA-ANC)
- TensorRT INT8 PTQ workflow: developer.nvidia.com TensorRT Developer Guide; NVIDIA/TensorRT GitHub #3978
- MIL-STD-810H ruggedization standard: crystalrugged.com, estonetech.com, betalight-tactical.com
- iDEX/DISC/SPARK program context: idex.gov.in; DRDO Dare to Dream / TDF scheme documentation
- ITU-T PESQ withdrawal / POLQA status: itu.int/rec/t-rec-p.862; [arXiv:2505.19760](https://arxiv.org/abs/2505.19760) ("Navigating PESQ")
- MELPe / STANAG 4591 military vocoder: rapidm.com, vocal.com, melpe.org (TSVCIS)
- MIL-STD-1474E noise limits / AHAH hearing hazard model: publications.aiha.org; hearingprotect.com; ResearchGate (MIL-STD-1474E design criteria)
- Jetson power management ([nvpmodel/jetson_clocks](https://nvpmodel.nvidia.com/jetson_clocks)): docs.nvidia.com Jetson Linux Developer Guide; forums.developer.nvidia.com
- Real-time audio I/O (PortAudio/ALSA/JACK): portaudio.com; [python-rtmixer docs](https://python-rtmixer.github.io); wiki.linuxaudio.org